Octopus CXL Memory Pooling: RL Results, Training System, and Next Steps

Pulak Mehrotra

April 2026

Three takeaways.

1. **RL is the only policy near break-even on both held-out traces.**

   Greedy fails on both traces (ratio 3.77 and 12.97). RL achieves pooling ratio 0.98 and 1.06 — at or below break-even.

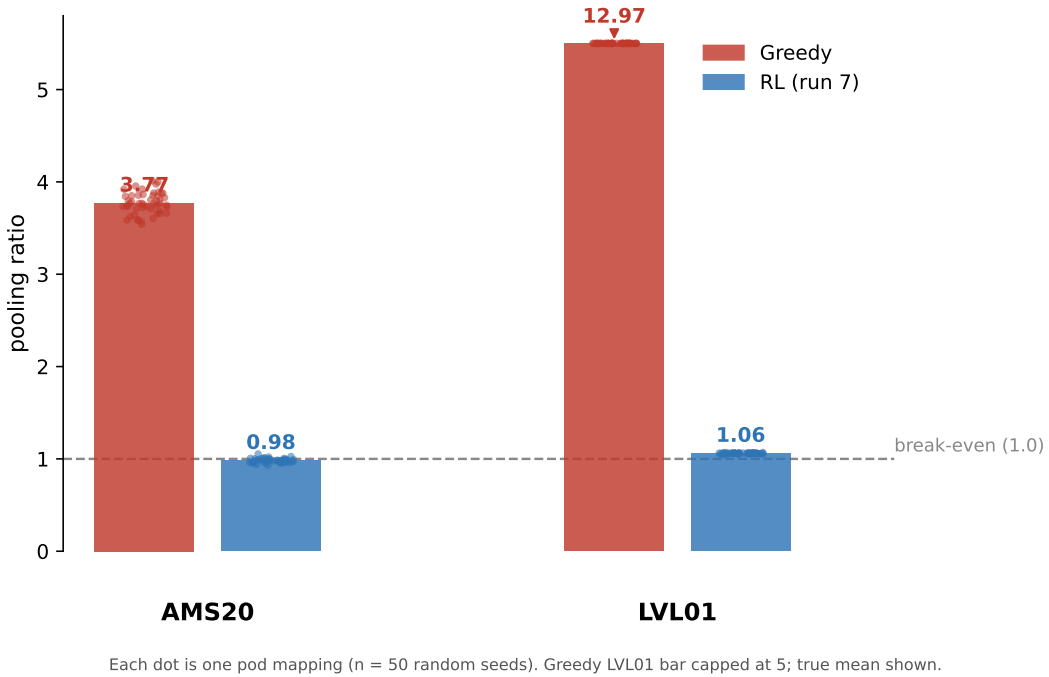
2. **The training system is not the bottleneck.**

   Data augmentation across 7 traces and 32 parallel environments solved generalisation and eliminated policy collapse.

3. **The remaining ceiling is a reward design problem.**

   Savings plateau at $\sim$1–2% within 100k steps and do not improve over 2M steps. The agent learns to game the proxy reward rather than maximise true pooling savings.

RL is the only policy near break-even on both held-out traces.

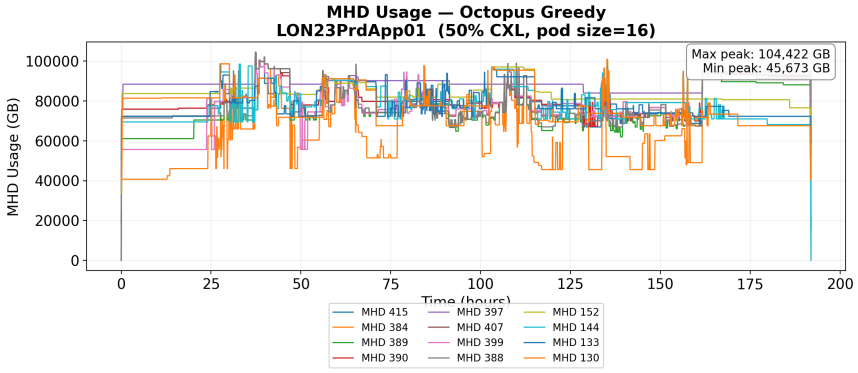


Each dot is one pod mapping (n = 50 random seeds). Greedy LVL01 bar capped at 5; true mean shown.

- Both test traces were **unseen during training**.
- Dots show all 50 pod mappings — results are **stable**.
- n=50 random pod mappings at 100% CXL fraction (production ≈65%).

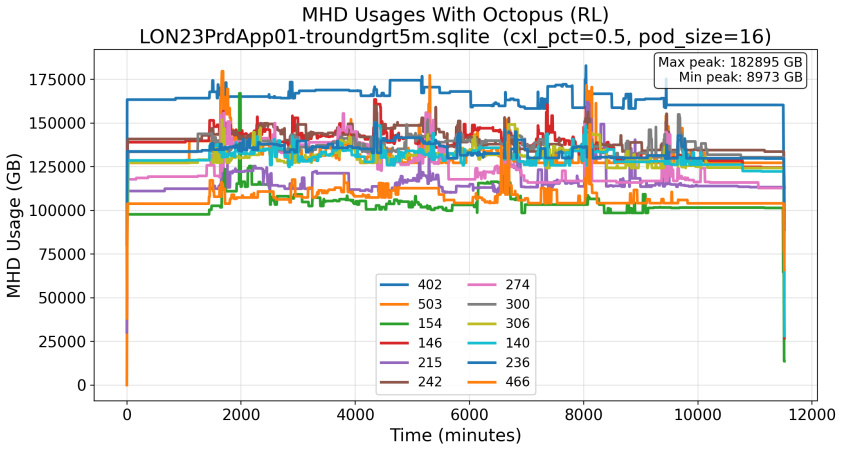
Greedy packs load onto a few MPDs; RL spreads it.

Greedy pooling ratio ≈ 6.1



A few MPDs saturate; most stay idle.

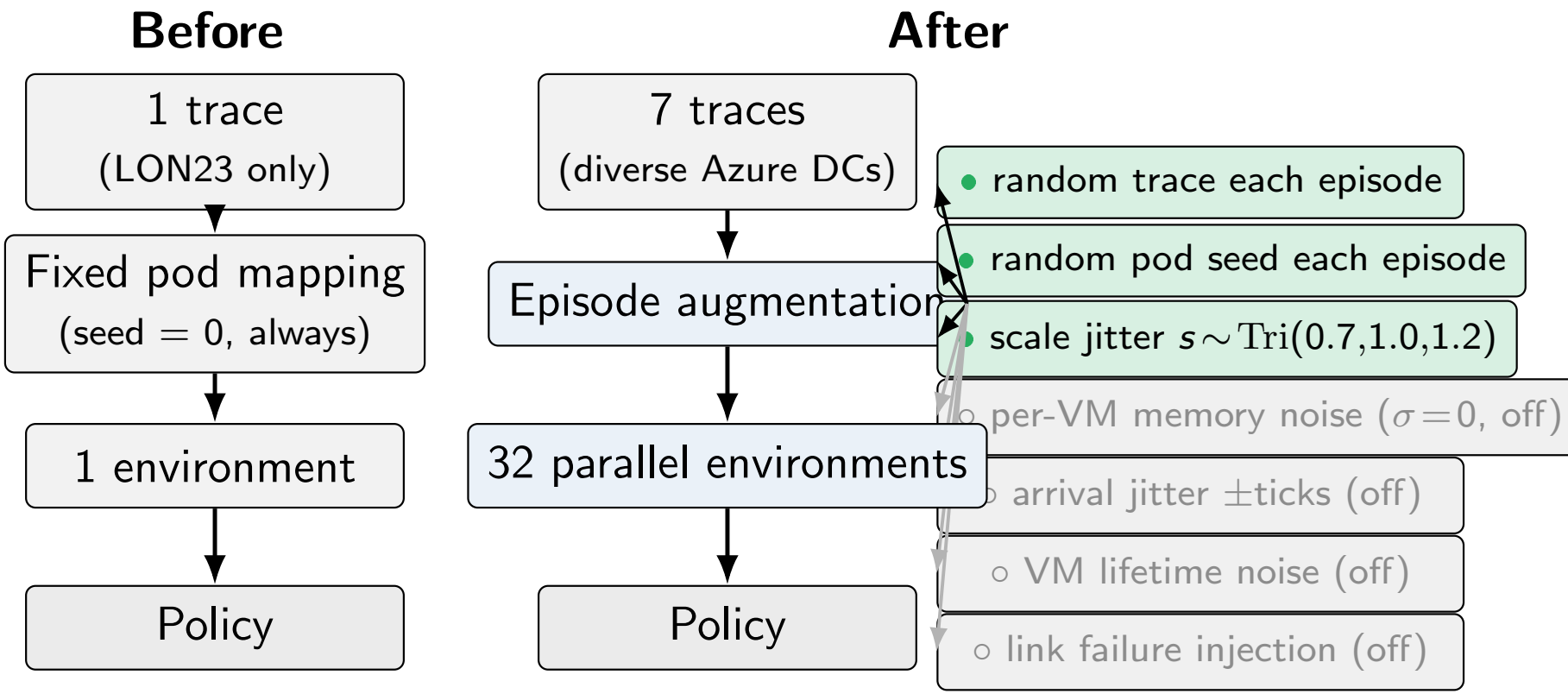
RL (run 7) pooling ratio ≈ 1.04



Load spread across all MPDs; no bottleneck.

LON23 (validation trace), CXL fraction 0.5, pod of 16 nodes. Each line = one MPD's load over time.

Data augmentation solved generalisation.



Memorises one DC.
Fails on new traces.

Generalises to AMS20 & LVL01 (never seen in training).

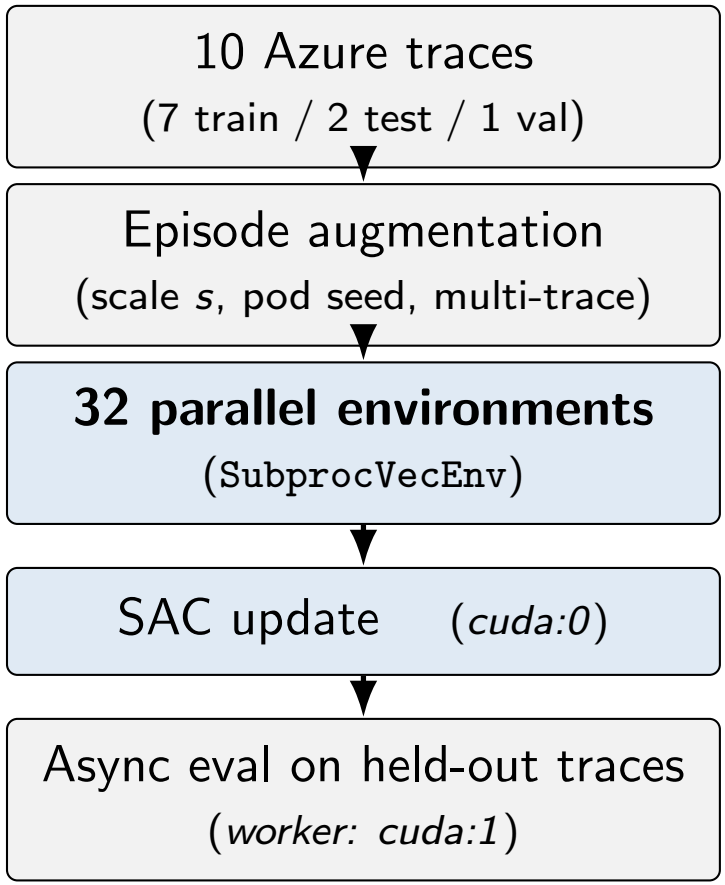
32 parallel environments cut rollout time by 26×.

Throughput

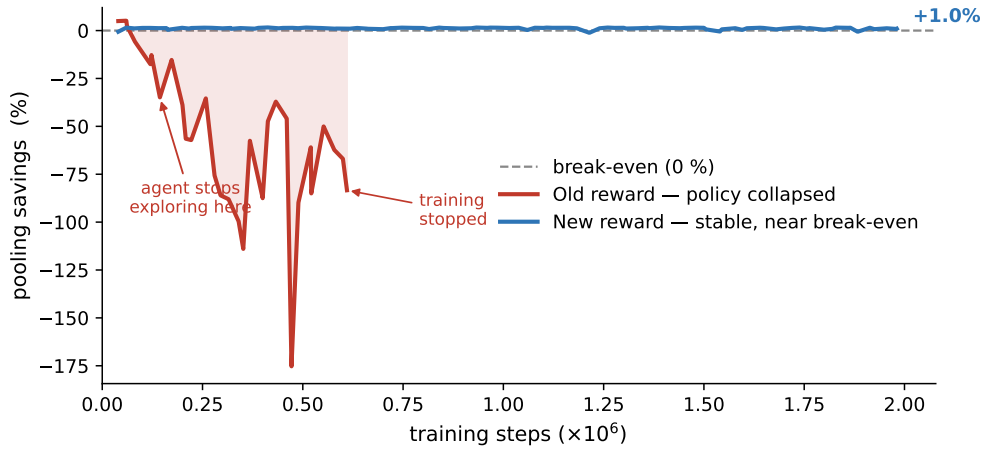
- Bottleneck: CPU rollout, not GPU.
- `DummyVecEnv(1)`: 46 steps/s
- `SubprocVecEnv(32)`: 1,221 steps/s
- **26× speedup.**
- 2M steps ≈ 14 min wall-clock.

SAC hyperparameters

- $\gamma = 0.999$, replay 1M, batch 256
- α auto-tuned
- Net: [256, 256]



The reward told the agent the wrong thing.



What went wrong

- The reward only watched the **single most-loaded MPD**.
- The agent found a shortcut: stop exploring and repeat one safe assignment.
- Pooling gets **worse**, not better.

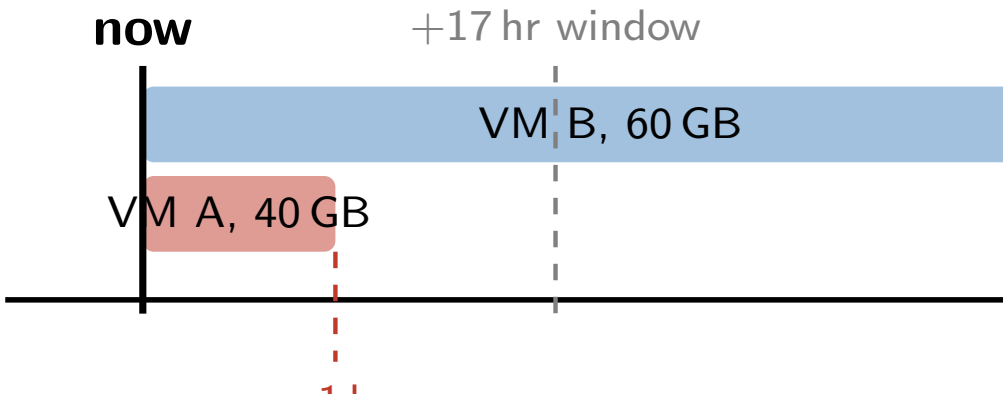
Fix: score all MPDs, and discount VMs that are about to leave.

Two changes to the reward

1. **Score all reachable MPDs** every step, not just the worst one. Every MPD now gives the agent feedback.

2. **Discount short-lived VMs.** A VM leaving in 1 hr should barely count against an MPD's load. A VM staying 2 days should count fully.

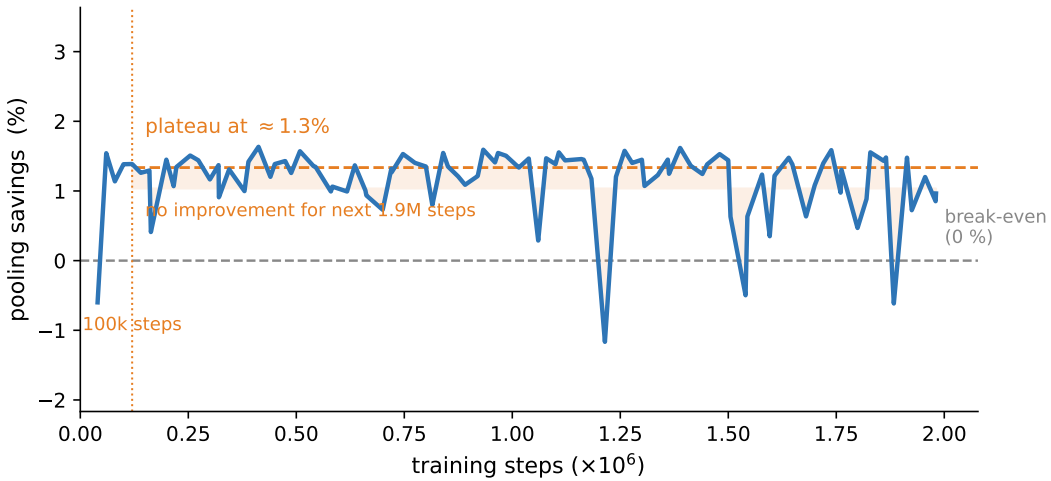
Result: stable training over 2M steps, near break-even on both held-out traces.



Raw load = 100 GB. **Sticky load ≈ 62**

VM A is nearly gone — it barely counts.

The new reward is stable — but stuck at 1–2%.

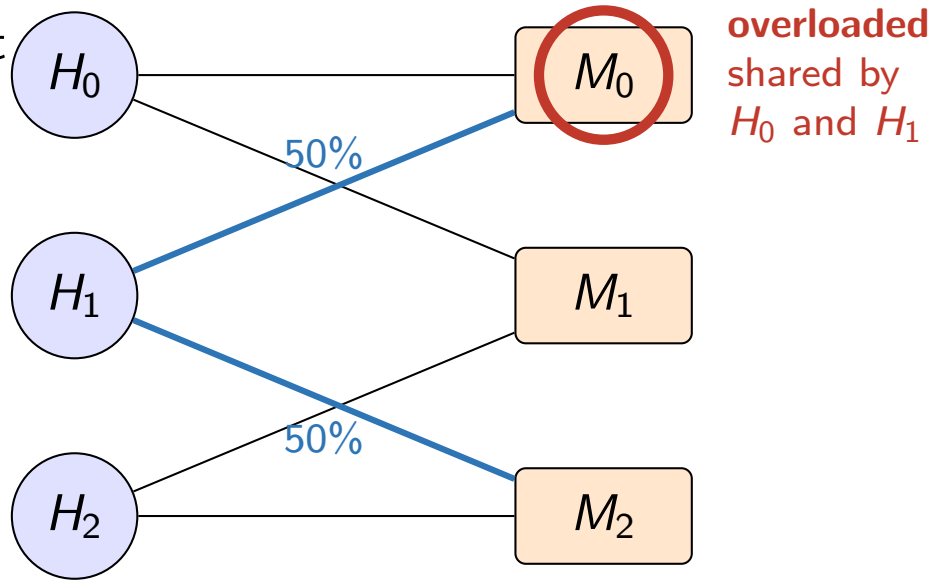


- Savings reach ≈1–2% within 100k steps, then **stop improving**.
- Training for 20× longer changes nothing.
- **More training is not the fix.** The agent has learned to game the reward signal, not to maximise real savings.

The next slides explain why, and what reward changes we plan to try.

The current reward is better — but it still misses the big picture.

- **What the current reward does:** it gives a better score when a host avoids leaving a lot of long-lasting load on the MPDs it can use.
- **Why that helped:** this removed the collapse from the old reward and made training much more stable.
- **What it still misses:** it judges each host's choice mostly by looking at that host's own neighborhood.
- So a host can make its own score look better by splitting load evenly, even if that sends more traffic onto an MPD that is already crowded for the whole pod.



What should the reward say instead?

- Score the agent on the **whole pod outcome**, not just whether one host made a locally good-looking choice; use **final pooling savings** as the main objective.
- Keep a small amount of step-by-step guidance with **potential-based reward shaping (PBRS)** so learning is easier, but the goal stays the same.
- Add an **optimal-gap** signal from the exact solver: tell the agent how far each placement is from the best possible choice for that snapshot.